AI ASSISTANT CODING
ASSIGNMENT-19.2

AI Assisted Coding

Name : K. Shiva Charan

H.no : 2303A52160

Batch : 34

Course Code : 23CS002PC304

Assignment Number : 19.2

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

• Prompt AI to translate a Python script that reads from and writes to a file into Java.

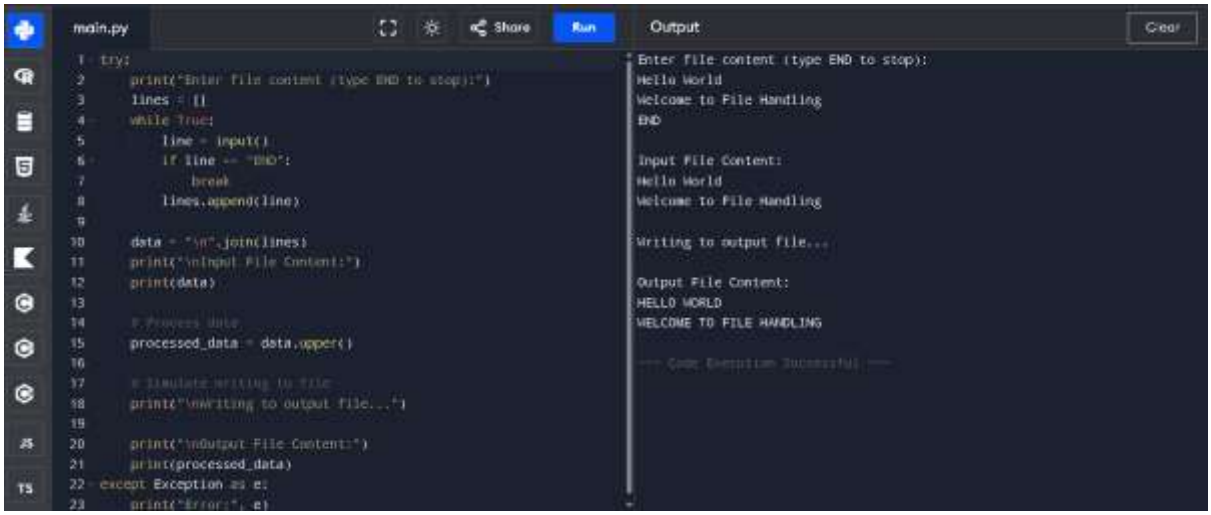
• Ensure proper handling of file streams, exceptions, and input/output formatting.

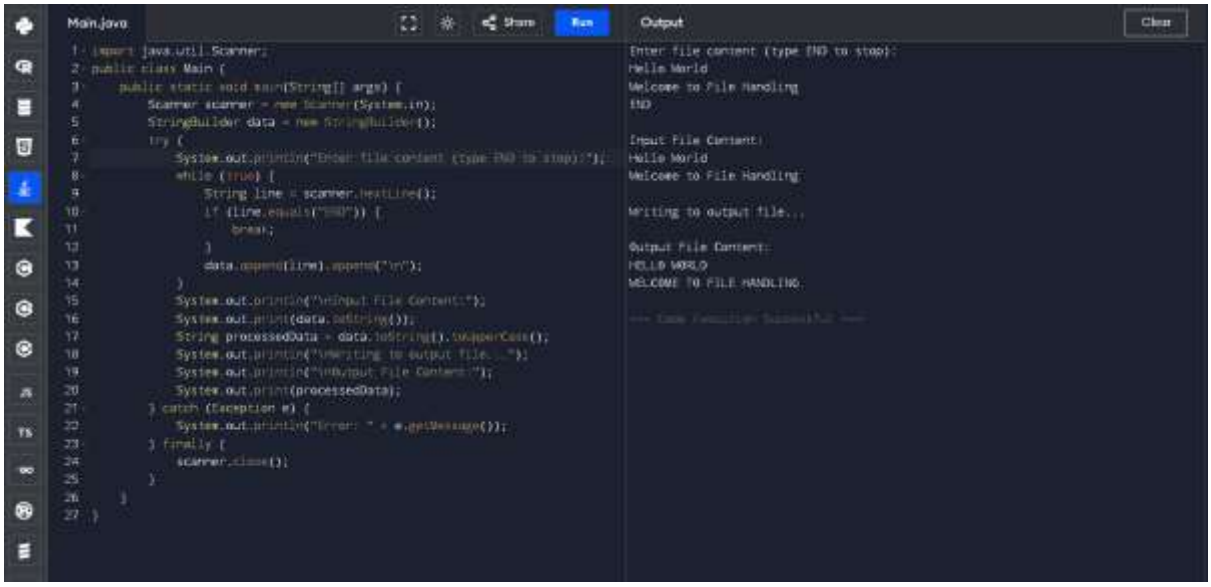
• Verify that the Java program produces the same file content and console output as the Python version.

• Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.





Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

• Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).

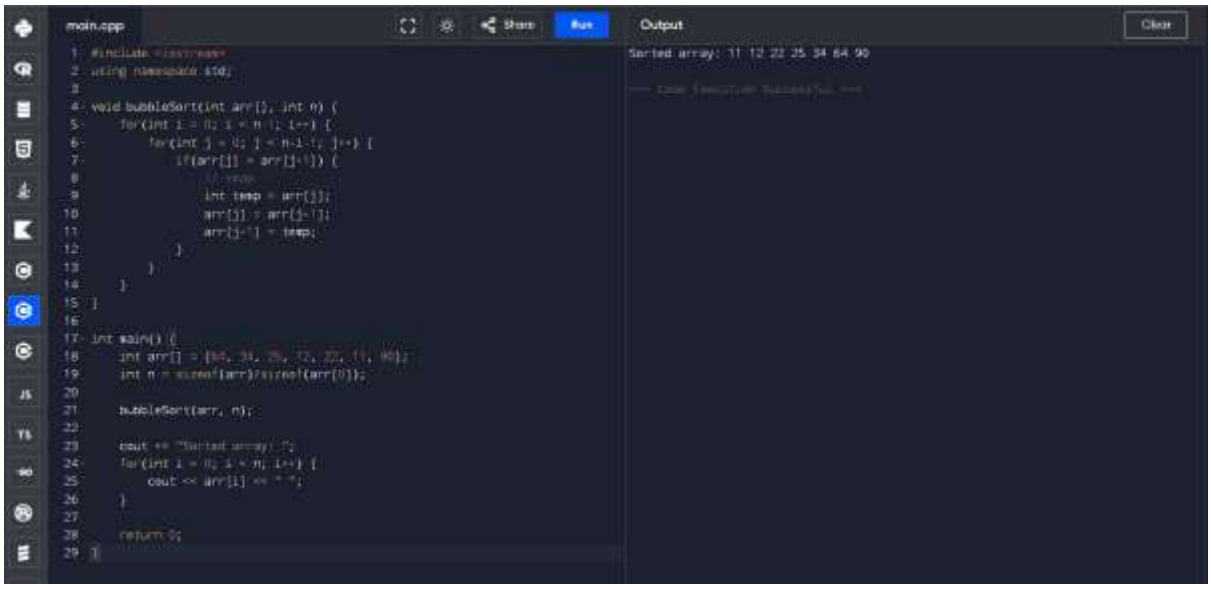
• Ask AI to generate an equivalent Python implementation.

• Ensure logical correctness and validate results using multiple test cases.

• Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.



Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

• Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).

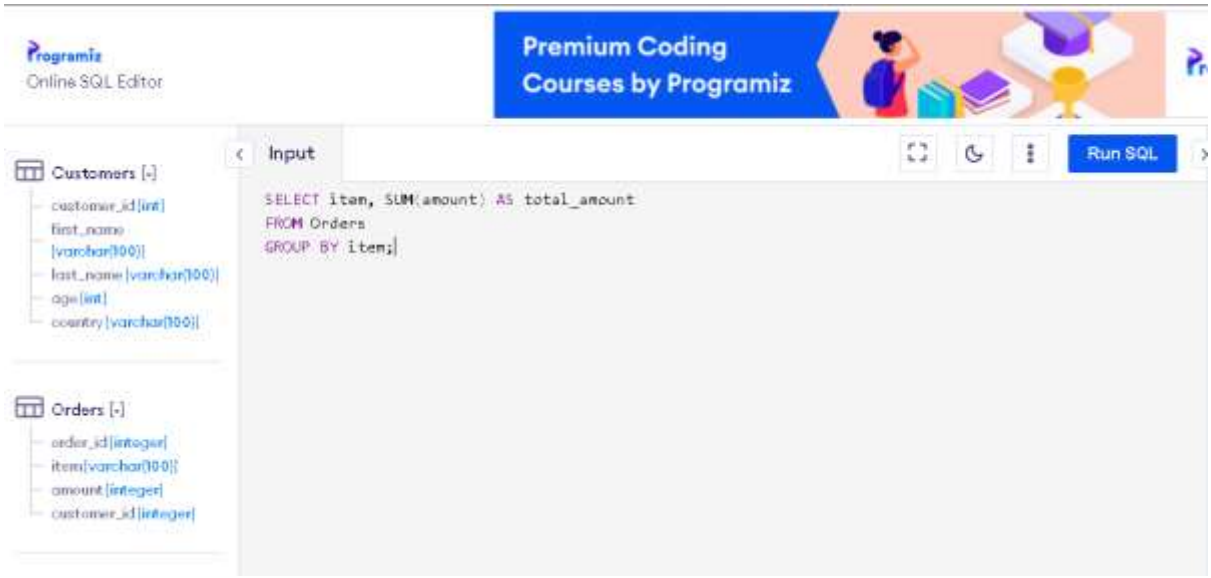
• Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.

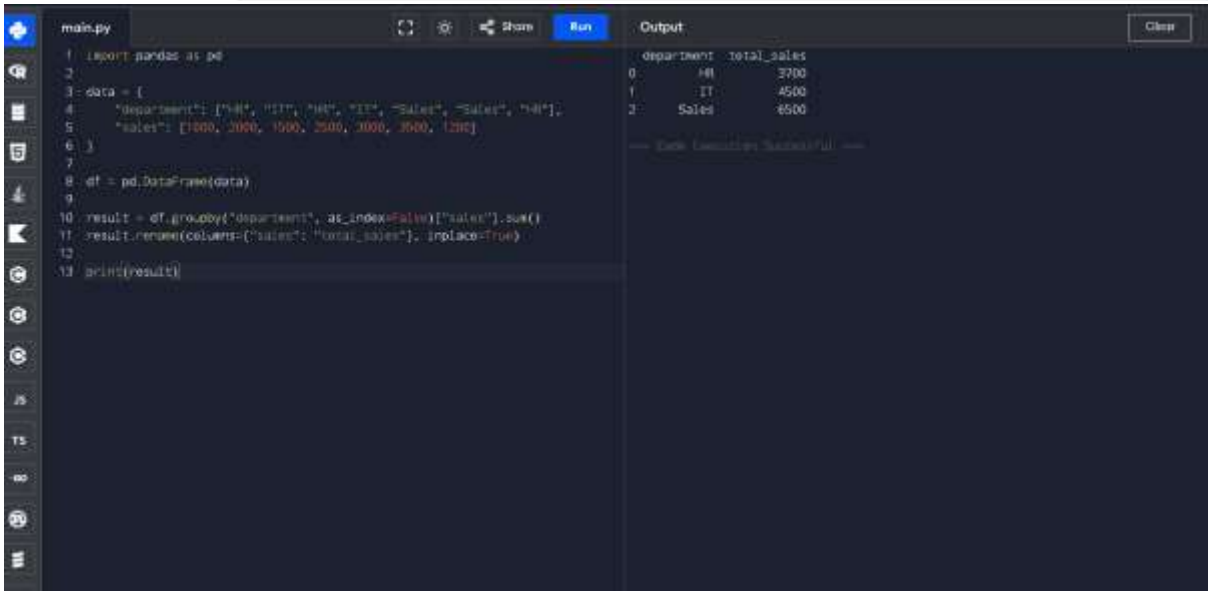
• Test the generated code using a sample dataset.

• Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.





Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

• Converting a REST API implementation from Python (Flask) to Java (Spring Boot).

• Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to API"

@app.route('/sum', methods=['GET'])
def calculate_sum():
    a = int(request.args.get('a'))
    b = int(request.args.get('b'))
    return jsonify({"result": a + b})
if __name__ == '__main__':
    app.run(debug=True)
```

```
Requirement already satisfied: itsdangerous>=2.2.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.0.2)
Requirement already satisfied: werkzeug>=3.1.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.1.6)
Requirement already satisfied: colorama in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from click>=8.1.3->flask) (0.4.6)
PS C:\Users\hp>
fwd-i-search: _
```

127.0.0.1:5000

Welcome to API

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
pair<vector<int>, int> processNumbers(vector<int> nums) {
    sort(nums.begin(), nums.end());
    int total = 0;
    for (int n : nums) {
        total += n;
    }
    return {nums, total};
}
int main() {
    vector<int> nums;
    int x;

    cout << "Enter numbers: ";
    while (cin >> x) {
        nums.push_back(x);
        if (cin.peek() == '\n') break;
    }
    auto result = processNumbers(nums);
    cout << "Sorted: ";
    for (int n : result.first) {
        cout << n << " ";
    }
    cout << "Sum: " << result.second;
    return 0;
}
```

Output

```
Enter numbers: 5 2 9 1
Sorted: 1 2 5 9
Sum: 17

=== Code Execution Successful ===
```

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

• Provide AI with a basic Python function that determines if a number is even or odd.

• Prompt AI to generate equivalent C++ code.

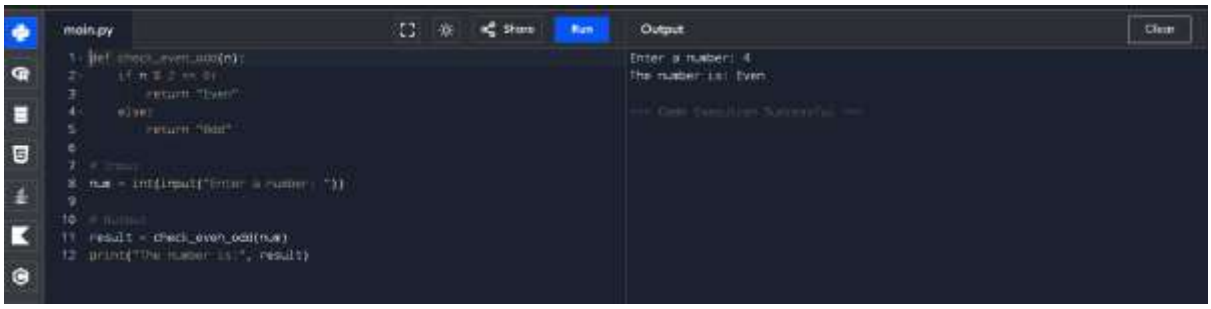
• Ensure correct syntax, input handling, and output formatting.

• Compile and run the C++ program to verify it produces the same result as the Python version.

• Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.



main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;

// Function to check even or odd
string checkEvenOdd(int n) {
    if (n % 2 == 0)
        return "Even";
    else
        return "Odd";
}

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    string result = checkEvenOdd(num);

    cout << "The number is: " << result << endl;

    return 0;
}
```

Output

Clear

```
Enter a number: 11
The number is: Odd


=== Code Execution Successful ===
```